

Lab Instructions

In this lab, we focus on **parallel AI inference and evaluation**: sharding a test set, running predictions with threads/processes, gathering predictions in the correct order, and comparing accuracy/runtime against a serial baseline.

The lab uses **scikit-learn digits** because it is small, reliable, and runs on most student laptops. The same structure also transfers to image classifiers, recommendation systems, and LLM request-routing simulations.

Before the lab, watch this video to learn PyTorch a bit.

<https://www.youtube.com/watch?v=r1bquDz5GGA&t=2405s>

Full Example Code: July27_parallel_inference.py

What Students Should Observe

Students may observe that **serial batch prediction is already very fast** for the digits dataset. This is intentional: it creates a useful teaching moment. Vectorized/batched inference may outperform manual Python-level parallelism because model internals and NumPy/scikit-learn code may already be optimized.

Students may also observe that process-based inference is slower than expected. This can happen because sending the model and chunks to worker processes creates overhead. Python's ProcessPoolExecutor runs work in separate processes, but input/output objects must be communicated between processes, and Executor.map() uses chunking behavior for process pools. [\[youtube.com\]](https://www.youtube.com/watch?v=r1bquDz5GGA&t=2405s)

Student Results Table

Students should produce a table like this:

Method	Workers	Chunk Size	Mean Time	Speedup	Samples/sec	Accuracy	Correct
serial	1	0		1.00			True
threaded	2	16					

Method	Workers	Chunk Size	Mean Time	Speedup	Samples/sec	Accuracy	Correct
threaded	4	64					
threaded	8	256					
process	2	16					
process	4	64					
process	8	256					

Discussion Questions

Ask students:

1. Which method was fastest?
2. Did parallelism help or hurt?
3. Did chunk size matter?
4. Did the predictions match the serial baseline?
5. Did accuracy change?
6. Why might process-based inference be slower here?
7. What would change if:
 - the dataset were 100× larger?
 - the model were much larger?
 - loading each input required file I/O?
 - prediction involved an external API?
8. What final project could use sharded inference?

Important Tips

1. Accuracy should not change

If all approaches use the same trained model and the same input samples, then accuracy should be the same. If accuracy changes, students likely mixed up prediction order or lost samples during aggregation.

2. Correctness comes before speed

Students should verify:

```
np.array_equal(parallel_pred, serial_pred)
```

If this is False, they should not report speedup yet.

3. Parallel inference can be slower

Parallel inference may be slower if:

- the dataset is small,
- chunks are too small,
- model prediction is already vectorized,
- the model is copied/pickled repeatedly,
- the machine has limited cores,
- nested parallelism causes oversubscription.

scikit-learn documents that some estimators and utilities can use joblib, OpenMP, or BLAS parallelism internally, and `n_jobs` controls joblib-managed parallelism for estimators that expose it. [\[uwpc-pyth....github.io\]](http://uwpc-pyth....github.io)

Extension A: Simulate Larger Inference Workload

The digits dataset is small. To make the inference workload larger, students can repeat the test set:

```
def enlarge_dataset(X, y, repeat=100):
```

```
    X_big = np.tile(X, (repeat, 1))
```

```
    y_big = np.tile(y, repeat)
```

```
    return X_big, y_big
```

```
X_test_big, y_test_big = enlarge_dataset(X_test, y_test, repeat=100)
```

Then benchmark on X_test_big.

This makes sharding more meaningful and may reveal different performance behavior.

Extension B: Add Artificial Preprocessing Cost

Students can simulate a CPU-heavy preprocessing step before prediction:

```
def expensive_preprocess(X_chunk):  
    """  
    Simulate CPU-heavy preprocessing.  
    """  
  
    X_new = X_chunk.copy()  
  
    for _ in range(100):  
        X_new = (X_new * 1.0001 + 0.0001) % 16.0  
  
    return X_new  
  
def predict_chunk_with_preprocess(args):  
    model, chunk = args  
  
    X_processed = expensive_preprocess(chunk["X"])  
    preds = model.predict(X_processed)  
  
    return {  
        "chunk_id": chunk["chunk_id"],
```

```
"start": chunk["start"],
"end": chunk["end"],
"preds": preds,
}
```

Then change the threaded/process workers to call:

```
predict_chunk_with_preprocess
```

This extension helps students see when process-based parallelism may become more useful.

Extension C: Latency Simulation

For students interested in LLM serving or request routing, simulate individual inference requests:

```
import time

import random

from concurrent.futures import ThreadPoolExecutor, as_completed


def simulated_request(request_id):
    """
    Simulate a request with variable latency.
    """

    start = time.perf_counter()

    # Simulated input preparation
    time.sleep(random.uniform(0.01, 0.04))

    # Simulated model inference
```

```
time.sleep(random.uniform(0.02, 0.08))
```

```
end = time.perf_counter()
```

```
return {  
    "request_id": request_id,  
    "latency": end - start,  
}
```

```
def simulate_request_serving(num_requests=100, workers=8):
```

```
    results = []
```

```
    start_all = time.perf_counter()
```

```
    with ThreadPoolExecutor(max_workers=workers) as executor:
```

```
        futures = [  
            executor.submit(simulated_request, i)  
            for i in range(num_requests)  
        ]
```

```
        for future in as_completed(futures):  
            results.append(future.result())
```

```
    total_time = time.perf_counter() - start_all
```

```
    latencies = [r["latency"] for r in results]
```

```

return {
    "num_requests": num_requests,
    "workers": workers,
    "total_time": total_time,
    "throughput": num_requests / total_time,
    "mean_latency": float(np.mean(latencies)),
    "p50_latency": float(np.percentile(latencies, 50)),
    "p95_latency": float(np.percentile(latencies, 95)),
}

```

```

for workers in [1, 2, 4, 8, 16]:
    result = simulate_request_serving(
        num_requests=100,
        workers=workers,
    )
    print(result)

```

This is an excellent bridge to final projects involving **LLM inference simulation**, queues, or real-time AI pipelines.

Extension D: PyTorch Inference Pattern

For teams using PyTorch, emphasize `model.eval()` and disabling gradient tracking.

```
import torch
```

```

def torch_predict(model, dataloader, device):
    model.eval()

```

```
all_preds = []

with torch.inference_mode():
    for batch in dataloader:
        inputs = batch["x"].to(device)

        logits = model(inputs)
        preds = logits.argmax(dim=1)

        all_preds.append(preds.cpu())

    return torch.cat(all_preds).numpy()
```

PyTorch's `no_grad` disables gradient calculation and can reduce memory consumption during inference. PyTorch's `inference_mode` is analogous to `no_grad` but removes additional overhead, while still requiring `model.eval()` for proper inference behavior such as disabling dropout and using batch normalization running statistics. [[scientific....github.io](https://scientific-github.io)] [github.com]

Deliverables

Each student will submit the following to my email, lidongvtemail@gmail.com.

1. Results Table

Must include:

- method
- workers
- chunk size
- runtime

- speedup
- samples/sec
- accuracy
- correctness

2. Short Reflection

Suggested prompt:

In 1–2 paragraphs:

1. Which method was fastest?
2. Did parallelism help or hurt?
3. Did prediction correctness match the serial baseline?
4. What bottleneck do you suspect?
5. How would this inference pattern help your final project?